ARGUS Demo Script

Walkthrough, queries, codebase

ARGUS Demo Script

What the demo shows, what to say, what to type, and what the codebase looks like when someone asks.

Runtime: **8 minutes** for the full walkthrough, **90 seconds** for the short version if a judge is passing through.

The 30-second version

> Every security camera makes you choose: send your footage to a company and get a system that improves, or keep it private and get one that never learns. ARGUS does both. Video is processed and stored entirely on this device — nothing leaves it. But every two weeks it sends what it *learned* to a shared model, as numbers, never images. And because we do not control the devices contributing, we built the part that makes that safe: screening that stops one broken device from poisoning everyone's model.

Part 1 — Detection, live (3 min)

Open the dashboard. Point at the badges before anything else: **on-device, frame rate, which accelerator is live, which models loaded.**

> Everything you are about to see runs on this board. No internet connection is involved in any of it.

1.1 Walk into frame

Say what is happening as the boxes appear:

> Motion fires first — it is nearly free, and it is the gate. Only when something moves do the expensive models run. A still room costs us almost nothing.
>
> Now the object model has found a person. That green box is the person class. And because it found a person, *now* face recognition runs — it is the most expensive stage, so it never runs speculatively.

1.2 Enrol a face

Type a name, click **Enrol**. Then step out and back in.

> That is a green box with a name. The system is not storing a photograph of me — it stored a 512-number mathematical description. You cannot reconstruct my face from it.

Have an unenrolled person step in.

> Red box, labelled UNKNOWN. Note it shows no confidence number. That is deliberate — the number would describe how sure we are that a *face* is present, not any claim about who it is, and showing it next to "unknown" invites exactly that misreading.

1.3 The threat class

Hold up the prop.

> Orange. Different model — a second network trained specifically on people carrying weapons. It runs on the whole frame, not on the person box, and there is a measured reason for that I will come back to.

1.4 The full classification

Have this table on the poster:

Class	Colour	Source	What triggers it
`MOTION`	not drawn	Frame differencing	Pixels changed
`HUMAN`	Green	YOLOv8n	A person
`VEHICLE`	Green	YOLOv8n	Car, truck, bus, motorcycle
`ANIMAL`	Green	YOLOv8n	Cat, dog, horse, sheep, cow
`UNKNOWN`	Green	YOLOv8n	Any of 69 other objects
`FACE`	Green / **Red**	ArcFace	Green = recognised, red = not
`DANGEROUS_PERSON`	**Orange**	YOLO11s	Person with a weapon-like object

> Seven classes, three separate models. Motion boxes are deliberately not drawn — they cover most of the frame and would bury everything else.

Part 2 — Recording (1 min)

> The cameras run continuously. The disk does not.

Show `/api/recordings` or the clips list.

> We only write footage around actual detections. Three details make the clips usable: the clip starts *before* the trigger, because a clip that begins the instant you are spotted has already missed you walking up. It continues after, so you do not get cut off mid-stride. And someone standing around produces one clip, not three hundred.
>
> Motion deliberately does not trigger recording. It fires on clouds and moving branches — recording on that is 24/7 recording with extra steps.

Part 3 — Federated learning (3 min)

This is the part that is research rather than assembly. Spend the time.

3.1 The problem

> To improve a security model the normal way, you collect everyone's footage centrally and train on it. That means shipping video of people's homes to a company.

3.2 The inversion

Draw or point at:

```
   Day 0        every device has the same model
   Days 1–13    each device learns from its own camera
                nothing is transmitted
   Day 14       each device sends only MODEL CHANGES — numbers
   server       combines them, sends back an improved model
```

> The model travels to the data instead of the data travelling to the model. No image ever crosses the network.

Show it:

```
curl http://<host>:8000/api/federated/status
```

```
{"enabled": true, "interval_days": 14,
 "last_round_at": "2026-09-01T02:00:00",
 "next_due_at": "2026-09-15T02:00:00", "idle": true}
```

> The next-due time matters. "It runs every two weeks" is unfalsifiable without it.

An aside worth having ready if someone asks how it survives a reboot:

> The schedule is written to disk. With an in-memory timer and a 14-day window, any reboot restarts the countdown — a device rebooted weekly would run federated learning *never*, and look perfectly healthy doing it.

3.3 The contribution

> Here is the problem nobody mentions. Standard federated averaging combines every update it receives. One device that is broken or hostile moves the shared model — and that damaged model goes back out to everyone. One bad contributor degrades the whole fleet.

Four filters, on the poster:

Filter	Catches
Structural	Malformed data, invalid numbers
Magnitude	Updates far larger than peers — trying to overwrite the model
Direction	Updates opposing everyone else — deliberate sabotage
Trimmed mean	Remaining outliers

> Then a final check: we score the candidate model, and if it is worse than the one it would replace, we throw it away and keep the old one.

Two lines that land with a technical judge:

> We compare model *changes*, not model weights. Raw weights are dominated by the shared starting point, so two updates that disagree completely still look almost identical. Comparing changes is what makes disagreement visible at all.
>
> And consensus uses the median, not the mean — because the mean is exactly what an attacker is trying to drag.

3.4 Say the limitation out loud

> Two of those filters compare devices against each other, so they need at least three contributors. With two devices that disagree, there is no way to know which one is wrong. The system detects that, reports it, and falls back to the checks that work regardless. We are not claiming the full defence runs on a two-node demo.

Volunteering this is worth more than being caught on it.

Part 4 — Privacy and security (1 min)

Leaves the device	Never leaves the device
Model weight arrays	Camera frames
Sample counts	Recorded clips
Loss and accuracy	Face images
A device ID	Face embeddings

> Face identity never participates in federated learning at all. Face data is not in the model that federates, so it cannot be in an update.

If asked about storage, answer straight:

> Federated learning means data does not *leave* the device. It does not mean nothing is stored. Each device keeps training crops and clips locally, on hardware the customer owns. "We never store your footage" would be false. "Your footage never leaves your device" is true.

Queries — every command, in demo order

Setup

```
python scripts/fetch_models.py                    # threat weights, checksum-verified
python scripts/export_hailo.py --hw-arch hailo10h # on an x86-64 Linux host, NOT the Pi
python -m pytest tests/ -q                        # 249 passed
python main.py
```

Health and status

```
curl localhost:8000/api/status
curl localhost:8000/api/detection/status
```

```
{"status":"running","fps":12.4,"backend":"hailo","motion_detection":true,
 "object_detection":true,"face_recognition":true,"threat_detection":true,
 "known_faces":3}
```

Field	If it is wrong
`"backend": "cpu"`	No Hailo export found — accelerator is idle
`"face_recognition": false`	InsightFace failed to load
`"threat_detection": false`	Threat model missing or ultralytics absent
`"fps": 0.0`	Fewer than two frames processed yet

Cameras

```
curl localhost:8000/api/cameras
curl -X POST localhost:8000/api/cameras/init
curl localhost:8000/api/cameras/front/status
# browser: http://<host>:8000/api/cameras/front/stream
```

Faces

```
curl localhost:8000/api/faces
curl -X POST "localhost:8000/api/faces?name=Giovanny"
curl -X DELETE localhost:8000/api/faces/<id>
curl -X POST localhost:8000/api/faces/reset      # between judges
```

> Reset between visitors. Over a multi-hour showcase the known-faces database fills with passers-by, and an unpruned database eventually resolves the *wrong* identity in front of someone — worse than recognising nobody.

Recordings and federation

```
curl localhost:8000/api/recordings
curl localhost:8000/api/federated/status
```

Authentication (only when exposed)

```
AUTH_REQUIRED=true ARGUS_ADMIN_PASSWORD='...' python main.py
curl -X POST localhost:8000/api/auth/login \
     -H 'Content-Type: application/json' \
     -d '{"username":"admin","password":"..."}'
curl -H "Authorization: Bearer <token>" localhost:8000/api/faces
scripts/run_tunnel.sh --check
```

Reproducing the numbers

```
python scripts/benchmark_classes.py --dataset <split>
python scripts/tune_threat_threshold.py --dataset <split> --negatives <dir>
python scripts/verify_enrollment.py --name "Giovanny"
```

If the demo breaks

Symptom	Do this
Threat stage misbehaving	`THREAT_ENABLED=false` and restart — returns to previous behaviour, no code change
Frame rate sagging	Raise `detect_every` on the stream URL
Wrong face matching	`POST /api/faces/reset`
Camera dead	`POST /api/cameras/init`; check `/api/cameras`
Everything wrong	Play the backup video and keep talking

Codebase breakdown

≈10,700 lines across 48 files. Tests are the largest single body of code, which is the point: on a demo day the failures that matter are the silent ones.

Area	Lines	Files	What it does
`tests/`	2,959	15	249 tests, all passing
`src/detection/`	1,960	8	Cascade, faces, threat, overlay, recording
`scripts/`	1,183	7	Model fetch, exports, benchmarks, tunnel
`src/api/`	1,095	4	38 endpoints, auth
`src/federated/`	1,049	6	Client, scheduler, robust aggregation
`src/camera/`	850	3	Platform detection, capture backends
`src/training/`	552	2	Local dataset, fine-tuning
`static/`	502	1	Dashboard, single file, no build step
`src/database/`	286	1	SQLite, six tables
`src/security/`	222	1	Hashing, tokens

Where to look for each claim

Claim	File
Cascade, gating, 7 classes	`src/detection/__init__.py`
Face recognition	`src/detection/face_recognition.py`
Threat detection	`src/detection/threat.py`
Event recording	`src/detection/event_recorder.py`
Robust aggregation	`src/federated/robust.py`
14-day schedule, persistence	`src/federated/scheduler.py`
Board detection, capture	`src/camera/platform_detect.py` , `backends.py`
Authentication	`src/api/auth.py`
Adversarial tests	`tests/test_robust_aggregation.py`

Test coverage

Suite	Tests
Face recognition + API	35
Platform + multi-camera	35
Authentication	33
Threat stage	30
Federated scheduling	23
Robust aggregation (adversarial)	**21**
Event recording	18
Detection cascade	11
Dashboard, streaming, other	43
Total	**249**

Questions to expect

"How accurate is it?"

> Face recognition is 99.96% verification accuracy on LFW, the standard benchmark — a one-in-ten-thousand false accept rate over 178,000 impostor pairs. Threat detection is 92.6% precision: when it flags someone, it is right more than nine times in ten. Recall is 0.80, so about one armed person in five is missed. It is a review aid, not a guard.

"What is your accuracy number?"

> For threat detection, be careful with that word — our test set is 80% positive, so a model that fires on every frame scores 0.796 without looking at anything. We report balanced accuracy, which is 0.5 for that baseline; ours is 0.773. The number worth quoting is precision, 0.926.

"Why not just use the cloud?"

> Because the price of cloud intelligence is your footage. We wanted both properties, and federated learning is how you get them.

"Could someone poison your model?"

> That is the exact problem we worked on — and yes, against plain federated averaging, trivially. We have a test that demonstrates the attack succeeding without our screening and failing with it.

"Why did you not use the fastest settings?"

> We tuned the threat threshold *up*, giving away recall. At the default, nearly one ordinary scene in three flagged somebody as armed. Pointed at a visitor, that is a credibility and an ethics problem.

"What does it not do?"

> No night vision. No signed updates, so we defend against bad updates but not forged identities. No encryption at rest yet. And the robust aggregation is tested but not yet wired into the live server.

"Is this your own model?"

> The object and face models are standard architectures. The threat model is a third-party graduation thesis artifact — attributed in `NOTICE.md`, and the reason the project is AGPL-3.0. Our contribution is the integration, the measurement, and the aggregation defence.